HypG: A Safe-Rust Post-Quantum Cryptographic Suite

White Paper — Design, Architecture, and Evaluation of the v2.0 Research Profile
(KEM v2.0; Symm/Hash v2.0; DS v2.0.1)

HypG Project
Independent research prototype

October 2026

Abstract

HypG is a from-scratch, zero-external-dependency cryptographic suite written in Rust with `#![forbid(unsafe_code)]` in every library. It comprises a Module-LWE key-encapsulation mechanism, a portable BLAKE3 hash/XOF/keyed-hash layer, a ChaCha20 plus keyed-BLAKE3 authenticated-encryption scheme (one-shot and streaming), and a stateless hash-based signature scheme. The v2.0 profile uses module rank four over $\mathbb{Z}_{4096}[X]/(X^{256}+1)$ for the KEM and an SLH-DSA-256f-shaped hypertree with FORS 35×9 for signatures. The signature wire format carries the separate **2.0.1** revision: explicit PRF/F domain tags and FIPS-205-style odd-node promotion in the WOTS+ L-tree.

This paper documents the design rationale, formal construction, implementation discipline, controlled evaluation, and—most importantly—the limits of the evidence. The recorded release has 146 deterministic tests passing in debug, release, and on the Rust 1.70 MSRV; four frozen KAT files totalling 234,114 bytes; a 14-suite adversarial harness; a 20,000-trial failure sweep; a full 11,520-bit sweep of one test ciphertext; Miri coverage of the KEM, AEAD, and DS bounds tests; and byte-identical KAT output on x86-64 and `qemu-aarch64`. None of this is a security proof. The KEM's exact FO-style transform is not covered by a published theorem, the lattice hardness of the chosen parameters has not been confirmed by an external estimator, the signature composition has no exact signing-volume bound, and there is no external audit, formal verification, `dudect`-grade timing study, or power/EM/fault campaign. HypG v2.0 is a research prototype and must not be deployed to protect real assets.

Keywords: post-quantum cryptography; Module-LWE; Fujisaki–Okamoto; hash-based signatures; FORS; WOTS+; XMSS; BLAKE3; ChaCha20; authenticated encryption; safe Rust; reproducible evaluation.

Document status. Technical report accompanying the v2.0 artifact. It is not peer-reviewed, not an audit, and not a security certification.

Contents

1 Introduction **4**
- 1.1 Contributions . . . 4
- 1.2 What this paper does not claim . . . 5
- 1.3 Organisation . . . 5

2 Background and related work **5**
- 2.1 Post-quantum cryptography . . . 5
- 2.2 Module-LWE . . . 5
- 2.3 Hash-based signatures . . . 5
- 2.4 Symmetric primitives . . . 5

3 Design principles **6**

4 Architecture **6**
4.1 Component graph . . . 6
4.2 Version map . . . 6
4.3 Boundary decisions . . . 6

5 The Module-LWE KEM **7**
5.1 Ring and parameters . . . 7
5.2 Key generation and encapsulation . . . 7
5.3 Decapsulation and implicit rejection . . . 8
5.4 Correctness . . . 8
5.5 Intended security argument and its gaps . . . 8

6 The CA-PRF and hash layer **9**
6.1 CA-PRF construction . . . 9
6.2 BLAKE3 API and framing . . . 9

7 Authenticated encryption **9**
7.1 One-shot construction . . . 9
7.2 Streaming front . . . 10
7.3 Security position . . . 10

8 Hash-based signatures **10**
8.1 Profile and keys . . . 10
8.2 Deterministic randomizer and digest . . . 10
8.3 Tweakable hash and v2.0.1 tags . . . 10
8.4 Trees . . . 11
8.5 Encoding and verification . . . 11
8.6 Security position . . . 11

9 Implementation assurance **11**
9.1 Memory and allocation discipline . . . 11
9.2 Secret lifetimes . . . 11
9.3 Constant-time posture . . . 12
9.4 Failure semantics . . . 12

10 Evaluation **12**
10.1 Method and evidence levels . . . 12
10.2 Correctness and reproducibility . . . 12
10.3 Adversarial testing . . . 13
10.4 Miri and portability . . . 13
10.5 Performance . . . 13
10.6 Interpretation . . . 14

11 Security analysis and open problems **14**
11.1 Assumptions . . . 14
11.2 The exact FO variant . . . 14
11.3 The DS signing volume . . . 14
11.4 The AEAD composition . . . 14
11.5 Side channels and fault injection . . . 14
11.6 Operational assumptions . . . 14

12 Threats to validity **14**

13 Related work **15**

14 Road to a defensible profile **15**

15 Conclusion **16**

A Parameter register **16**

B Artifact and reproducibility notes **16**

1 Introduction

Public-key cryptography deployed today is vulnerable to a cryptographically relevant quantum computer: Shor's algorithm solves integer factorisation and discrete logarithms in polynomial time. Symmetric primitives are less affected, but their effective security halves under Grover search, so a post-quantum suite must combine quantum-resistant public-key constructions with conservative symmetric cryptography and a disciplined implementation.

HypG is an experiment in building the *whole* suite rather than a single primitive: key establishment, hashing, authenticated encryption, and signatures, sharing one small implementation core. The project imposes unusually strict engineering constraints: no external dependencies, no `unsafe` code, `no_std` cryptographic libraries, fixed public loop bounds, explicit little-endian encodings, deterministic test vectors, and mechanically checked CI gates.

The current release is **v2.0.0** with a **DS v2.0.1** wire revision. It is a research prototype. The documentation deliberately separates (i) what the code does, (ii) what the test suite establishes, and (iii) what would still be required for a security claim. This paper follows the same separation.

1.1 Contributions

The contributions are:

1. **A complete, dependency-free implementation.** A `no_std`, no-allocation cryptographic core and four protocol pillars, with all dependency edges confined to workspace path dependencies and all library crates under `#![forbid(unsafe_code)]`.

2. **A precise v2.0 parameter profile.** A rank-four Module-LWE KEM at lattice dimension 1024 with a power-of-two modulus $q = 4096$, and an SLH-DSA-256f-shaped signature profile with $n = 32$, $h = 68$, $d = 17$, $h' = 4$, FORS $(35, 9)$, and WOTS+ $w = 16$.

3. **A documented DS 2.0.1 revision.** Explicit one-byte PRF/F tags remove a structural collision between the two 64-byte keyed-hash inputs, and odd WOTS+ L-tree nodes are promoted unchanged in the FIPS 205 style.

4. **An honest treatment of the CA-PRF.** The cellular automaton is presented as a branchless mixing and domain-separation stage whose uniformity argument rests on the final BLAKE3 XOF, not as an independently proven hardness assumption.

5. **A corrected KEM correctness analysis.** The compression residuals from c_1 and c_2 are included explicitly, yielding a working standard deviation of approximately 70 and a margin of approximately 14.6σ against the 1024 decision boundary. This is a heuristic estimate, not a formal failure bound.

6. **A byte-identical streaming AEAD.** The streaming front uses the same subkeys, MAC input, counter schedule, and tag as the one-shot construction; only the scheduling differs.

7. **A reproducible evaluation record.** Frozen KAT checksums, cross-architecture equality, a deterministic red-team harness, a local lattice lab, Miri subsets, and a controlled release benchmark are recorded with commands and machine details.

8. **Explicit non-claims and an audit agenda.** The paper names the exact proof, estimator, side-channel, and operational gaps that stand between the current artifact and a deployable system.

1.2 What this paper does not claim

This paper does *not* claim that HypG is IND-CCA2 secure, UF-CMA/SUF-CMA secure, or a secure AEAD in the sense of a published reduction for its exact composition. It does not claim NIST Category-5 security, FIPS compliance, certification, or production readiness. It does not claim constant-time execution at the machine-code level, and it does not claim resistance to power, electromagnetic, or fault attacks. KAT vectors pin behaviour; they are not security evidence.

1.3 Organisation

Section 2 reviews the relevant background. Section 3 states the design constraints. Section 4 describes the architecture. Sections 5–8 describe the four pillars. Section 9 discusses implementation assurance. Section 10 records the controlled evaluation. Section 11 analyses assumptions and open problems. Section 12 discusses threats to validity. Section 13 relates HypG to prior work. Section 14 gives the path to a defensible profile.

2 Background and related work

2.1 Post-quantum cryptography

The NIST post-quantum standardisation process produced ML-KEM (FIPS 203) [1], ML-DSA (FIPS 204) [2], and SLH-DSA (FIPS 205) [3]. ML-KEM is a Module-LWE KEM; ML-DSA is a Module-LWE/Module-SIS signature scheme; SLH-DSA is a stateless hash-based signature scheme derived from SPHINCS+ [8]. HypG does not implement any of these standards verbatim. It uses the same broad mathematical families and some of the same dimension choices, but with different primitives, encodings, domain separation, and wire formats.

2.2 Module-LWE

Module-LWE generalises ring-LWE by working over R_q^k. For a public matrix A and secret/error vectors s, e drawn from a small distribution, the public key is $b = As + e$. Encryption uses a fresh small vector r and errors e_1, e_2:

$$c_1 = A^T r + e_1, \qquad c_2 = \langle b, r \rangle + e_2 + \mathsf{Encode}(m).$$

The decryption noise is $\langle e, r \rangle + e_2 - \langle s, e_1 \rangle$; correctness requires it to remain inside the decoding region. The concrete hardness depends on the ring degree, module rank, modulus, noise distribution, and number of samples. The community lattice-estimator [10] is the standard tool for estimating the cost of known attacks; it has not been run on HypG's exact parameters.

2.3 Hash-based signatures

SLH-DSA and SPHINCS+ combine a few-time FORS signature with a hypertree of XMSS/WOTS+ one-time signatures. The hash function is tweaked by a public seed and an address so that every hash call is domain-separated. HypG follows this architecture with BLAKE3 as the tweakable hash, a custom 32-byte address, and a deterministic randomizer. Determinism is useful for reproducible vectors but removes fresh-randomness hedging; the security consequences are discussed in Section 8.

2.4 Symmetric primitives

BLAKE3 [4] is a tree-structured hash and XOF with a 32-byte keyed mode and a reference `derive_key` mode. ChaCha20 [5] is a 20-round ARX stream cipher with a 96-bit nonce and a 32-bit block counter. HypG uses both as designed: BLAKE3 for hashing, expansion, key

derivation, and MACs; ChaCha20 for confidentiality. The AEAD is an Encrypt-then-MAC composition rather than a standardised AEAD algorithm.

3 Design principles

Five constraints shape every component.

1. **No external dependencies.** Every dependency edge is a workspace path dependency. The supply-chain surface is therefore the Rust toolchain and the repository itself, not a transitive crate graph. `cargo tree` is part of the release gate.

2. **No `unsafe`.** Every library crate uses `#![forbid(unsafe_code)]`. Arithmetic and conversion paths are written to be total where possible. The cost is scalar code and best-effort, rather than volatile, zeroization.

3. **`no_std` cryptographic libraries.** The core, KEM, Symm, AEAD, DS, and hash crates do not require an allocator, a filesystem, or an operating system. Threads, files, and allocation live only in `std` crates.

4. **Fixed public geometry.** Protocol loops have public bounds. Secret-dependent branches, secret-dependent array indices, and secret-dependent early returns are avoided in the cryptographic paths.

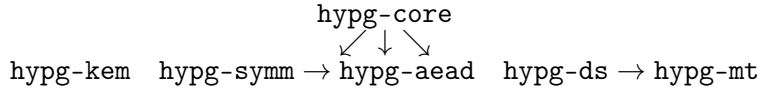
5. **Reproducible evidence.** Deterministic KATs, a frozen checksum inventory, an adversarial harness, a lattice lab, cross-architecture equality, and a recorded benchmark are part of the artifact.

These are engineering constraints, not security proofs. In particular, source-level branchlessness does not imply machine-code constant-time execution, and safe Rust does not imply resistance to microarchitectural leakage.

4 Architecture

4.1 Component graph

The dependency graph is acyclic:



`hypg-core` supplies portable BLAKE3, ChaCha20, the CA-PRF expander, polynomial arithmetic over $\mathcal{R}_q$, and constant-time helpers. `hypg-hash` is a thin API over the core hash functions. `hypg-kat`, `hypg-bench`, `hypg-redteam`, `hypg-lattice`, and `hypg-cli` are `std` harnesses or operator tools.

4.2 Version map

4.3 Boundary decisions

Threading is deliberately excluded from the `no_std` crates. `hypg-mt` uses `std::thread::scope` and owns all worker results; deterministic assembly makes its output byte-identical to the serial signer. SIMD intrinsics are excluded by the `unsafe` policy and portable SIMD is not stable on the MSRV; the implementation is scalar and relies on the compiler only for ordinary code generation.

Component	Code version	Wire/spec revision	Frozen vectors
Workspace	2.0.0	—	—
`hypg-kem`	2.0.0	KEM v2.0	`kats/v2.0.0.txt`
`hypg-symm`	2.0.0	Symm v2.0	`kats-symm/v2.0.0.txt`
`hypg-hash`	2.0.0	Hash v2.0	`kats-hash/v2.0.0.txt`
`hypg-ds`	2.0.0	**DS v2.0.1**	`kats-ds/v2.0.1.txt`

Table 1: Version identity of the active artifacts. The literal `HYP-G/v07/...` strings are frozen domain separators, not a statement that the parameter set is v0.7.

5 The Module-LWE KEM

5.1 Ring and parameters

Let

$$\mathcal{R}_q = \mathbb{Z}_q[X]/(X^{256}+1), \qquad q = 2^{12} = 4096,$$

and let $k = 4$. Secrets and errors are drawn from CBD(2), whose coefficients lie in $\{-2,-1,0,1,2\}$ with probabilities $1/16, 4/16, 6/16, 4/16, 1/16$ and variance one. The message is 32 bytes, encoded one bit per coefficient at spacing $q/2 = 2048$. The public key is

$$pk = \mathit{seed}_A(32) \,\|\, \mathsf{Pack}_{12}(b_0) \,\|\, \cdots \,\|\, \mathsf{Pack}_{12}(b_3),$$

the ciphertext is

$$ct = \mathsf{Pack}_{10}(\mathrm{Compress}_{10}(c_{1,0})) \,\|\, \cdots \,\|\, \mathsf{Pack}_{10}(\mathrm{Compress}_{10}(c_{1,3})) \,\|\, \mathsf{Pack}_5(\mathrm{Compress}_5(c_2)),$$

and the secret key is $sk = \mathit{seed} \,\|\, \mathit{fake} \,\|\, pk$. The byte sizes are 1568, 1440, 1632, and 32 for pk, ct, sk, ss.

5.2 Key generation and encapsulation

Key generation derives seed_A from the master seed with the CA-PRF, expands a 4×4 matrix A from it, expands s and e from the master seed, and computes $b = As + e$. A separate CA-PRF domain derives the 32-byte implicit-rejection key *fake*.

Encapsulation treats the caller-provided 32-byte `coins` argument as the FO message m:

$$\mathit{rseed} = \mathsf{H}(m \,\|\, pk \,\|\, \texttt{"HYP-G/v07/r-seed"}),$$

then derives (r, e_1, e_2) from *rseed* using the `rand-r`, `rand-e1`, and `rand-e2` domains. It computes

$$c_1 = A^T r + e_1, \qquad c_2 = \langle b, r \rangle + e_2 + \mathsf{Encode}(m),$$

packs the ciphertext, and derives

$$ss = \mathsf{XOF}(\texttt{"HYP-G/v07/ss"} \,\|\, m \,\|\, ct \,\|\, \mathsf{H}(pk), 32).$$

API warning. The `coins` value is not an auxiliary nonce. It is the message itself and the input to the FO randomness generator. Reusing it under the same public key repeats both the ciphertext and the shared secret. It must be a fresh CSPRNG output for every encapsulation.

Component	Variance
$\langle e, r \rangle$	1024
$-\langle s, e_1 \rangle$	1024
e_2	1
$-\langle s, \Delta_1 \rangle$	1280
Δ_2	1365.25
Total	4694.25
Standard deviation	≈ 68.5

Table 2: Working variance model for the KEM noise. The independence and Gaussian approximations are not proven.

5.3 Decapsulation and implicit rejection

Decapsulation unpacks and decompresses (c_1, c_2), recomputes s from the secret seed, and obtains

$$v = c_2 - \langle s, c_1 \rangle, \qquad m' = \mathsf{Decode}(v).$$

It then recomputes the encapsulation randomness and the full ciphertext ct', compares ct' with the received ct using a full-scan constant-time equality, and computes

$$ss_{\text{ok}} = \mathsf{XOF}(\texttt{"}\ldots\texttt{/ss"} \,\|\, m' \,\|\, ct \,\|\, \mathsf{H}(pk), 32), \qquad ss_{\text{rej}} = \mathsf{XOF}(\texttt{"}\ldots\texttt{/reject"} \,\|\, \mathit{fake} \,\|\, ct, 32).$$

A branchless select returns ss_{ok} when the ciphertext matches and ss_{rej} otherwise. The function never returns an error. A protocol must not treat a returned 32-byte value as proof that the ciphertext was valid; it must confirm the shared secret through a subsequent authenticator or transcript binding.

5.4 Correctness

Ignoring compression, the decryption value is

$$c_2 - \langle s, c_1 \rangle = \mathsf{Encode}(m) + \langle e, r \rangle + e_2 - \langle s, e_1 \rangle.$$

Compression adds residuals Δ_1 on each c_1 coefficient and Δ_2 on c_2:

$$v' = \mathsf{Encode}(m) + \langle e, r \rangle + e_2 - \langle s, e_1 \rangle + \Delta_2 - \langle s, \Delta_1 \rangle.$$

The residuals satisfy $\Delta_1 \in \{-1, 0, 1, 2\}$ and $\Delta_2 \in \{-63, \ldots, 64\}$; over uniformly distributed coefficients their variances are 1.25 and 1365.25. A working variance model is:

Rounding up gives $\sigma \approx 70$ and a margin of $1024/70 \approx 14.6\sigma$ against the decision boundary. This is a heuristic estimate, not a formal tail bound. The decapsulation failure probability has not been bounded to 2^{-128}; the in-tree 32-trial unit test is a regression tripwire, not the argument. A formal analysis must bound the tail of the bounded-sum distribution directly, and a defensible empirical study must use at least 10^6 trials with a stated confidence interval.

5.5 Intended security argument and its gaps

The intended argument is FO-style: Module-LWE hides m for a passive adversary; the re-encryption check prevents ciphertext modification; the random-oracle derivation of (r, e_1, e_2) from (m, pk) prevents inconsistent randomness; implicit rejection removes a public validity oracle. The exact construction is not a verbatim copy of a standard FO transform. In particular, the accepted secret is $\mathsf{KDF}(m \,\|\, ct \,\|\, \mathsf{H}(pk))$ and the rejection value is $\mathsf{KDF}(\mathit{fake} \,\|\, ct)$. No cited theorem covers those exact choices, and no reduction is proved in this repository.

The lattice parameters have not been run through the community estimator. The power-of-two modulus differs from ML-KEM-1024's 3329, so ML-KEM cost estimates cannot be copied. Consequently the KEM is *not* claimed IND-CCA2 secure; the target is conditional on the missing analysis.

6 The CA-PRF and hash layer

6.1 CA-PRF construction

For a 32-byte seed x, domain d, and output length L:

$$\begin{aligned} \mathit{init} &= \mathsf{XOF}(x \,\|\, d \,\|\, \texttt{"-init"}, 256), \\ \mathit{rule} &= \mathsf{XOF}(x \,\|\, d, 64), \\ \mathit{state} &= \mathrm{CA}^{32}(\mathit{init}; \mathit{rule}), \\ \mathit{out} &= \mathsf{XOF}(d \,\|\, \mathit{state} \,\|\, d \,\|\, \texttt{"-out"}, L). \end{aligned}$$

The CA is a 32-word state with a fixed hyper-graph neighborhood (strides 1, 5, 17), a secret 8-word rule, and 32 branchless rounds. All loop bounds are public and all state is stack-resident.

The final BLAKE3 XOF is the security-relevant expander. If BLAKE3 is a secure XOF or random oracle, the output is pseudorandom regardless of the CA. The CA is therefore a design hedge and a deterministic mixing stage, not an independently justified hardness assumption. This distinction is important: the paper does not claim that the automaton adds provable security on top of BLAKE3.

6.2 BLAKE3 API and framing

The hash layer exposes unkeyed BLAKE3-256, a 64-byte XOF prefix, keyed BLAKE3, the reference `derive_key` construction, an expander $\mathsf{XOF}(\mathit{seed} \,\|\, \mathit{domain})$, a KDF $\mathsf{XOF}(\mathit{domain} \,\|\, \mathit{parts}\ldots)$, and a streaming hasher. `expand` and `kdf` do not insert length separators; every protocol caller uses fixed-length fields or a single variable-length field with a distinct domain. The KEM and CA-PRF callers satisfy this condition; a new caller must check it explicitly.

7 Authenticated encryption

7.1 One-shot construction

Let K be a 32-byte master key and N a 12-byte nonce. Derive

$$k_e = \mathrm{DK}(\texttt{"HYP-G/v07/symm-enc"}, K), \qquad k_m = \mathrm{DK}(\texttt{"HYP-G/v07/symm-mac"}, K).$$

Encrypt $C = \mathrm{ChaCha20}(k_e, N, 0, M)$ and authenticate

$$T = \mathrm{KeyedBLAKE3}(k_m, N \,\|\, AD \,\|\, C \,\|\, \mathrm{LE64}(|AD|) \,\|\, \mathrm{LE64}(|C|)).$$

The one-shot opener recomputes the tag and performs a full-scan constant-time comparison before writing any plaintext. A tag mismatch or length mismatch returns `None` and leaves the plaintext buffer untouched.

The message limit is exactly 2^{38} bytes: 2^{32} ChaCha20 blocks of 64 bytes. Counter exhaustion fails closed rather than wrapping. The nonce must be unique per $(K, \text{message})$; the construction has no nonce-misuse resistance.

7.2 Streaming front

The streaming sealer buffers a partial 64-byte keystream block across calls, feeds ciphertext bytes to a running keyed-BLAKE3 state, and appends the two little-endian length suffixes at `finish`. The opener feeds ciphertext to the MAC while releasing tentative plaintext; the plaintext becomes valid only when `finish` returns `Some(())`. On failure, the assembled plaintext buffer is zeroized. Empty chunks, one-byte chunks, block-boundary splits, and 1 MiB streams are tested. The streaming output is byte-identical to the one-shot output for the same inputs.

7.3 Security position

The EtM structure is standard, but the exact ChaCha20 + keyed-BLAKE3 composition is not a standardized AEAD and no reduction for this exact instantiation is included. The 32-byte tag is not truncated. The construction has no key-commitment guarantee; protocols requiring key commitment must add it. Nonce discipline is entirely the caller's responsibility.

8 Hash-based signatures

8.1 Profile and keys

The signature profile is $n = 32$, $h = 68$, $d = 17$, $h' = 4$, FORS $(k, a) = (35, 9)$, and WOTS+ $w = 16$ with $\ell = 67$. The public key is

$$pk_{\text{sig}} = root(32) \,\|\, pubseed(32),$$

and the secret key is

$$sk_{\text{sig}} = esk(32) \,\|\, prf(32) \,\|\, pubseed(32) \,\|\, root(32).$$

The master secret is *esk*. *pubseed* is derived from *esk* with the `ds-pubseed` domain, *prf* with `ds-prf`, and the root is the top XMSS layer root at layer 16, tree 0.

8.2 Deterministic randomizer and digest

The randomizer is

$$R = \mathsf{H}\big(\text{CA-PRF}(prf, \texttt{"HYP-G/v07/ds-addrnd"}, 64) \,\|\, \mathsf{H}(msg)\big).$$

There is no fresh per-signature entropy. Signing is a deterministic function of (sk, msg), so repeated signing returns the same bytes. This is intentional for KATs and reproducibility, but it leaks message equality and removes fresh-randomness fault hedging. The scheme is *not* described as freshly hedged.

The 64-byte digest is

$$md_c = \mathsf{H}(R \,\|\, pubseed \,\|\, root \,\|\, \mathsf{H}(msg) \,\|\, \text{LE32}(c) \,\|\, \texttt{"HYP-G/v07/ds-msghash"})$$

for $c \in \{0, 1\}$. The first 387 bits are split into 35 nine-bit FORS indices, a 68-bit tree index, and a four-bit bottom leaf. The remaining bits are unused.

8.3 Tweakable hash and v2.0.1 tags

All tweakable hashes are keyed BLAKE3 with *pubseed*:

$$\begin{aligned}
\mathsf{PRF}(pubseed, x, A) &= \text{KeyedBLAKE3}(pubseed, 01 \,\|\, x \,\|\, A),\\
F(pubseed, A, x) &= \text{KeyedBLAKE3}(pubseed, 02 \,\|\, A \,\|\, x),\\
\mathsf{H}(pubseed, A, L, R) &= \text{KeyedBLAKE3}(pubseed, A \,\|\, L \,\|\, R).
\end{aligned}$$

The explicit tags are the DS 2.0.1 revision. Before it, PRF and F both had 64-byte inputs and could be structurally identical when a secret seed happened to equal the address bytes. The tags make the first input byte different in every call. The 96-byte untagged H call has a distinct arity and field order.

8.4 Trees

FORS uses 35 trees of height 9. Each tree signs one nine-bit leaf index; the signature contains the selected secret leaf and its nine authentication nodes (320 bytes per tree). WOTS+ maps a 32-byte message to 64 nibbles plus a three-digit checksum, and signs by applying a 15-step chain; every chain executes all 15 steps with a branchless select so the secret digit does not control the loop trip count. The L-tree compresses 67 chain ends with seven levels; an unpaired final node is promoted unchanged. An XMSS layer is a height-4 Merkle tree over 16 WOTS+ leaves. The hypertree stacks 17 such layers; tree i is $tree_idx \gg 4(i+1)$, the bottom leaf is the digest's four-bit leaf, and upper leaf indices are the low four bits of the child tree index.

8.5 Encoding and verification

The 49,992-byte signature is

$$R(32) \,\|\, \mathrm{FORS}_{35} \,\|\, \mathrm{layer}_0 \,\|\, \cdots \,\|\, \mathrm{layer}_{16}.$$

Each layer is an eight-byte reserved zero field, a 67-element WOTS+ signature, and a four-node authentication path. Verification checks the exact length, rejects any nonzero reserved byte, recomputes the digest and FORS key, climbs all layers, and compares the resulting root with the public root using constant-time equality. No field is silently ignored.

8.6 Security position

The composition is SLH-DSA-shaped but not FIPS 205 conformant. It has no exact UF-CMA/SUF-CMA proof for its BLAKE3 instantiation, address layout, deterministic randomizer, or reserved-byte encoding. FORS is a few-time signature: security degrades as more signatures are produced under one key. The implementation enforces no signing limit and keeps no state. A deployment profile would need a concrete bound on signatures per key and a matching multi-target analysis. The current paper claims only that the implementation computes the specified bytes and that the deterministic vectors and tamper tests pass.

9 Implementation assurance

9.1 Memory and allocation discipline

The cryptographic libraries use fixed-size arrays and caller-provided buffers. They do not allocate, do not read the clock, do not access the filesystem, and do not spawn threads. The `std` crates own all OS interaction. CI greps the `no_std` libraries for `Vec`, `Box`, `alloc::`, `String`, and `std::` and fails if any appears.

9.2 Secret lifetimes

Secret-holding types redact `Debug`, avoid `Copy` where duplication would be dangerous, compare in constant time where equality is exposed, and best-effort zeroize on `Drop`. The types include the KEM secret key and shared secret, the DS secret key and pre-sign context, the AEAD key, the CA state and rule, the hash state, and polynomial buffers. This is best-effort erasure in safe Rust: normal writes are followed by compiler fences and a black-box use of the buffer pointer, but safe

Rust cannot guarantee volatile erasure. The accessors `seed_copy`, `fake_copy`, and `public_copy` return caller-owned copies that the caller must erase.

9.3 Constant-time posture

The implementation avoids secret-dependent branches and indices in the cryptographic paths: KEM decapsulation always performs re-encryption and both KDFs; WOTS+ chains always execute all 15 steps; tag and root comparisons scan all bytes; implicit-rejection selection is a byte mask. This is a source-level discipline, not a machine-code guarantee. Compiler transformations and microarchitectural effects are not analysed, and no `dudect` [11] or power/EM/fault campaign has been performed.

9.4 Failure semantics

The KEM never returns an error; invalid ciphertexts yield a pseudorandom secret. The one-shot AEAD returns `None` and leaves the output untouched on authentication failure. The streaming opener marks plaintext as tentative until `finish`. Signature verification returns `false` on any malformation. The CLI uses exit code 2 for usage/input errors and 3 for authentication failures.

10 Evaluation

10.1 Method and evidence levels

We use five evidence levels: E0 source inspection; E1 deterministic tests and KATs; E2 adversarial harnesses; E3 dynamic/portability analysis such as Miri and cross-architecture runs; E4 independent review. The current artifact has E0–E3 evidence and no E4 evidence. No result below is an external audit.

The controlled benchmark command was:

```
HYPG_BENCH_ITERS=8 \
HYPG_BENCH_DS_ITERS=8 \
HYPG_BENCH_TTEST_N=10000 \
cargo run -q --release -p hypg-bench
```

The machine was an Intel Core i5-14400F with 16 hardware threads and 31 GiB RAM, running Linux x86-64 with `rustc 1.97.1`. The benchmark warms up once and reports integer averages over eight iterations; large-message rates use three repetitions of 1 MiB.

10.2 Correctness and reproducibility

The debug, release, and MSRV 1.70 test suites each report **146 passed, 0 failed, 0 ignored**. The breakdown is 108 unit tests, 25 CLI negative integration tests, and 13 doc-tests. The red-team harness passes all 14 suites. The frozen KAT inventory is:

The complete SHA-256 values are:

```
kats/v2.0.0.txt
3b2dde3d1138c11b54a6880f50ca7b4390071ae53aa94601075f39b2af971d4b
kats-symm/v2.0.0.txt
91b7942ffb4a3c5da8b8b589268af7b7d1594796b428efdc7e26c567c5f04208
kats-hash/v2.0.0.txt
5fa778035ce2e8aa9c89345458ba4da67df4bdbe71a456cbb58026cf9341d9f1
kats-ds/v2.0.1.txt
e88c1bdf54e9390736cd03c1a165616d744a42c48d662880b344c7de3546c813
```

File	Bytes	SHA-256 (prefix)
`kats/v2.0.0.txt`	28,514	`3b2dde3d...71d4b`
`kats-symm/v2.0.0.txt`	3,662	`91b7942f...4208`
`kats-hash/v2.0.0.txt`	923	`5fa77803...d9f1`
`kats-ds/v2.0.1.txt`	201,015	`e88c1bdf...c813`
Total	234,114	

Table 3: Frozen vector inventory. The complete hashes are in `docs/parameters.md` and `docs/testplan.md`.

Operation	Controlled result
KEM keygen / encaps / decaps	795 / 931 / 1124 μs
BLAKE3 hash / streaming / XOF (1 MiB)	842 / 837 / 827 MB/s
ChaCha20 XOR (1 MiB)	773 MB/s
AEAD seal / open (1 MiB)	427 / 434 MB/s
DS sign / verify	72.009 ms / 3.725 ms
Threaded DS sign (16 workers)	18.329 ms, 3.928×

Table 4: Controlled release measurements on the record machine. These are local point estimates, not portable guarantees.

The x86-64 and `qemu-aarch64` static-musl KAT outputs are byte-identical across all 234,114 bytes. This checks endianness, integer width, and the DS 2.0.1 revision on an emulated ARM64 target; it is not native ARM64 validation.

10.3 Adversarial testing

The deterministic red-team harness covers garbage inputs, ciphertext malleability, an 8×8 key-mismatch matrix, message extremes, FO binding, sampler statistics over 524,288 CBD coefficients, failure sweeps, determinism, AEAD forgery, DS forgery, burst faults, tampered public keys, padding flips, a panic sweep, secret-key bit flips, and streaming AEAD games. A fresh release run with 20,000 trials reported 20,000/20,000 clean decapsulations. A separate full-bit run flipped all 11,520 bit positions of a test ciphertext; every flip changed the shared secret. These are regression and sanity checks, not a 2^{-128} failure proof.

10.4 Miri and portability

Miri is an interpreter for Rust's mid-level IR. Latest recorded runs on the hardened commit report KEM 9/9 (13,213.79 s), AEAD 6/6 (214.46 s), and DS bounds 3/3 (0.42 s). Full DS signing is not run under Miri because the hypertree is hours-long under interpretation. Miri checks executed paths for some classes of undefined behaviour; it does not prove their absence. The libraries also build for `aarch64`, `wasm32`, `s390x` (check only), and `thumbv7em`; KAT and red-team binaries pass under a 1 MB stack.

10.5 Performance

The valid-versus-tampered medians were 1004 per-mille for KEM decapsulation, 602 per-mille for AEAD open, and 994 per-mille for DS verify. The Welch t-test over 10,000 interleaved KEM samples had $\max |t| = 0.737$. These are timing-smoke signals, not a side-channel proof.

10.6 Interpretation

The evaluation establishes that the specified bytes are reproducible, that the tested failure paths behave as designed, that the KATs survive debug/release and cross-architecture comparison, and that the scalar implementation has usable research performance. It does not establish cryptographic security, constant-time execution, or production readiness.

11 Security analysis and open problems

11.1 Assumptions

The intended security argument depends on: (A1) Module-LWE hardness for the exact parameters; (A2) BLAKE3 as a collision-resistant hash, PRF/MAC, and XOF; (A3) ChaCha20 as a secure stream cipher with unique nonces; (A4) the exact FO-style transform having CCA security under A1–A2; (A5) random-oracle behaviour of the hash/KDF calls; (A6) a negligible decapsulation-failure probability; (A7) nonce uniqueness; (A8) a bounded DS signing volume; (A9) caller-protected secret storage; and (A10) no machine-code constant-time claim.

11.2 The exact FO variant

Standard FO analyses [6, 7] do not automatically cover the chosen shared-secret and rejection derivations. A proof must adapt the transform, account for the full public-key binding in G, and bound the re-encryption-collision probability. This is an open item, not a footnote.

11.3 The DS signing volume

FORS is few-time. The security of the hypertree depends on how many messages are signed under one key and on the probability of reusing FORS leaves or WOTS+ keys. The current implementation stores no state and enforces no limit. A deployment profile needs a concrete bound and a multi-target analysis; neither exists here.

11.4 The AEAD composition

The EtM composition is structurally standard, but the exact ChaCha20 + keyed-BLAKE3 instantiation has no cited reduction. Nonce reuse is catastrophic and caller-controlled. No key-commitment property is claimed.

11.5 Side channels and fault injection

Source-level branchlessness is not a machine-code guarantee. The project has no `dudect` evaluation, no power/EM laboratory, and no fault-injection campaign. The deterministic DS randomizer also removes fresh-randomness fault hedging. These are material limitations for any deployment-oriented claim.

11.6 Operational assumptions

The caller must supply a CSPRNG, protect keys in memory, ensure nonce uniqueness, confirm KEM secrets, bound signature volume, and provide protocol framing and replay protection. The libraries provide none of these services.

12 Threats to validity

Construct validity. KATs and reproducibility checks measure *behaviour*, not hardness. A deterministic test can pass while the construction is insecure.

Internal validity. The red-team harness is deterministic and finite. It does not cover every input, every protocol composition, or every compiler transformation. The 20,000-trial failure sweep is not a tail bound.

External validity. Performance is measured on one CPU, one toolchain, and one operating system. Cross-architecture KATs are emulated on ARM64; native ARM64 and s390x runtime validation are absent. The results do not establish portable timing behaviour.

Conclusion validity. The absence of external audit, lattice-estimator output, formal proof, and side-channel data prevents any conclusion about deployable security.

13 Related work

ML-KEM [1], ML-DSA [2], and SLH-DSA [3] are the NIST standards closest to the relevant families. HypG shares the Module-LWE and hypertree shapes but is not wire-compatible or standards-conformant. SPHINCS+ [8] and XMSS [9] are the main references for the signature composition. BLAKE3 [4], ChaCha20 [5], and the Fujisaki–Okamoto literature [6, 7] are the symmetric and transform references. The lattice-estimator [10] is the required external tool for the KEM parameters. The `dudect` [11] methodology is the minimum bar for a timing evaluation. Rust's Miri [12] is used for partial undefined-behaviour checking.

HypG's contribution is not a new hardness assumption. It is an integrated, transparent, reproducible research artifact with an explicit evidence boundary.

14 Road to a defensible profile

The shortest honest path to a deployable profile is:

1. Run the community lattice-estimator on the exact $(n, k, q, \eta) = (256, 4, 4096, 2)$ parameters, publish classical and quantum cost tables, and compare against ML-KEM-1024 without claiming equivalence.
2. Derive an analytic decapsulation-failure bound and measure it at at least 10^6 trials.
3. Prove or refute the exact FO-style transform, including the accepted and rejected secret derivations.
4. Prove a UF-CMA/SUF-CMA bound for the exact DS composition and choose an enforced signing-volume limit.
5. Complete a `dudect`-style timing evaluation and, if the threat model requires it, power/EM/fault evaluation.
6. Add coverage-guided fuzzing and a protocol-level misuse analysis.
7. Commission an independent code audit and publish findings.
8. If FIPS compliance is required, redesign around approved primitives and pursue ACVP/CAVP and CMVP validation.

Until then, HypG should remain in research, teaching, reproducibility, and test environments.

15 Conclusion

HypG v2.0 is a complete, zero-external-dependency, safe-Rust post-quantum research suite with a documented KEM, hash layer, AEAD, and stateless signature scheme. Its evaluation is unusually broad for a prototype: frozen KATs, cross-architecture equality, 146 tests, a 14-suite adversarial harness, Miri subsets, and controlled performance measurements. Its security claims are deliberately narrow: the artifact implements the specification, the tested paths behave as specified, and the remaining proof, estimator, side-channel, and audit work is named explicitly.

That combination—substantial engineering evidence and an explicit refusal to round it up into a security claim—is the contribution of this paper.

A Parameter register

Object	Value	Note
KEM ring	$n = 256$, $q = 4096$	$R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$
KEM rank	$k = 4$	lattice dimension 1024
KEM noise	CBD(2)	variance 1, support $\{-2, \ldots, 2\}$
KEM sizes	$pk = 1568$, $ct = 1440$, $sk = 1632$, $ss = 32$	bytes
DS hash	$n = 32$	BLAKE3-256
DS tree	$h = 68$, $d = 17$, $h' = 4$	17 XMSS layers
DS FORS	$k = 35$, $a = 9$	35 trees, 512 leaves each
DS WOTS+	$w = 16$, $\ell = 67$	64 message + 3 checksum digits
DS sizes	$pk = 64$, $sk = 128$, $sig = 49992$	bytes
AEAD	key 32, nonce 12, tag 32	bytes
AEAD limit	2^{38}	bytes per $(key, nonce)$
CA state	32 words of 64 bits	256 bytes
CA rounds	32	fixed

B Artifact and reproducibility notes

The active documentation is in `docs/`. The frozen vectors are under `crates/hypg-kat/`. The controlled benchmark command and machine details are in `docs/metrics.md`. The test matrix and evidence levels are in `docs/testplan.md`. The exact security non-claims are in `docs/threat-model.md` and `SECURITY.md`.

The core reproducibility commands are:

```
cargo test --workspace --locked
cargo test --workspace --locked --release
cargo +1.70 test --workspace --locked
cargo run -q -p hypg-redteam
cargo run -q -p hypg-lattice
cargo test -p hypg-kat --lib --locked
HYPG_BENCH_ITERS=8 HYPG_BENCH_DS_ITERS=8 HYPG_BENCH_TTEST_N=10000 \
  cargo run -q --release -p hypg-bench
```

The code is licensed under GPL-3.0-or-later. The paper sources are tracked in `papers/`; the PDFs are built with `pdflatex`.

References

[1] National Institute of Standards and Technology. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. FIPS 203, August 2024.

[2] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard*. FIPS 204, August 2024.

[3] National Institute of Standards and Technology. *Stateless Hash-Based Digital Signature Standard*. FIPS 205, August 2024.

[4] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn. BLAKE3: one function, fast everywhere. Technical report, 2020. https://github.com/BLAKE3-team/BLAKE3-specs/.

[5] Y. Nir and A. Langley. ChaCha20 and Poly1305 for IETF Protocols. RFC 8439, June 2018.

[6] E. Fujisaki and T. Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *CRYPTO 1999*, LNCS 1666, pages 537–554, 1999.

[7] D. Hofheinz, K. Hövelmanns, and E. Kiltz. A modular analysis of the Fujisaki–Okamoto transformation. In *TCC 2017*, LNCS 10677, pages 341–371, 2017.

[8] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe. The SPHINCS+ signature framework. In *ACM CCS 2019*, pages 2129–2146, 2019.

[9] A. Hülsing, D. Butin, S.-L. Gazdag, J. Rijneveld, and A. Mohaisen. XMSS: eXtended Merkle Signature Scheme. RFC 8391, May 2018.

[10] M. R. Albrecht, R. Player, and S. Scott. On the concrete hardness of Learning with Errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.

[11] O. Reparaz, J. Balasch, and I. Verbauwhede. Dude, is my code constant time? In *DATE 2017*, pages 1697–1702, 2017.

[12] The Rust Project. Miri: an interpreter for Rust's mid-level intermediate representation. https://github.com/rust-lang/miri.